Ubuntu-HPL

Created by	Kei Farouk
Category	ubuntu
Last updated time	@27 March 2026 02:26

Tutorial 3: Install, Compile and Run High Performance LinPACK (HPL) Benchmark

> **Sourced from:** CHPC SCC Github Repo

HPL is a crucial tool in the HPC community for benchmarking and comparing the performance of supercomputing systems. The benchmark is a software package designed to solve a dense system of linear equations using double-precision floating-point arithmetic. It is commonly used to measure the performance of supercomputers, providing a standardized way to assess their computational power.

You will now install and run HPL on your Virtual Machine.

---

1. Update the system and install dependencies

Install the tools needed to compile applications using `make`, a maths library for matrix operations, and `MPI` for parallel processes.

Ubuntu:

bash

```
sudo apt update
sudo apt install build-essential openmpi-bin libopenmpi-dev
libatlas-base-dev wget nano -y
```

---

2. Download HPL 2.3

Download the HPL 2.3 tarball from a reliable mirror, extract it, and move into the directory.

Download HPL Tarbel

```
wget https://fossies.org/linux/privat/hpl-2.3.tar.gz -O hpl
-2.3.tar.gz
```

Extract

```
tar -xzf hpl-2.3.tar.gz
```

Move to a nice folder name

```
mv hpl-2.3 ~/hpl
cd ~/hpl
```

See its contents

```
ls
```

---

3. Configure HPL

Copy and edit your own `Make.<YOUR_NAME>` file in the `hpl` directory to suit your system configuration.

```
cp setup/Make.Linux_PII_CBLAS_gm Make.<YOUR_NAME>
nano Make.<YOUR_NAME>
```

You need to carefully edit your `Make.<YOUR_NAME>` file, ensuring that you make the following changes:

Ubuntu based systems:

conf

```
ARCH          = <TEAM_NAME>

MPdir         = /usr/lib/x86_64-linux-gnu/openmpi

LAdir         = /usr/lib/x86_64-linux-gnu/atlas/
LAlib         = $(LAdir)/libblas.so $(LAdir)/liblapack.so

CC            = mpicc
LINKER        = mpicc
```

---

4. Temporarily edit your `PATH` variable

You are almost ready to compile HPL. You will need to modify your `PATH` variable in order for your MPI C Compiler `mpicc` to be a recognized binary.

```
# Temporarily append openmpi binary path to your PATH variable
# These settings will reset after you logout and re-login again
```

```
export PATH=/usr/lib/x86_64-linux-gnu/openmpi/bin:$PATH
which mpicc
```

---

5. Compile HPL

You are finally ready to compile HPL. Should you encounter any errors and need to make adjustments, first run `make clean arch=<TEAM_NAME>` before recompiling.

bash

```
make arch=<YOUR_NAME>
```

Confirm that your xhpl binary has been successfully built

```
ls bin/<YOUR_NAME>
```

---

6. Configure your `HPL.dat`

bash

```bash
cd bin/<YOUR_NAME>
nano HPL.dat
```

Carefully edit your `HPL.dat` file and verify the following changes:

conf

```conf
1            # of process grids (P x Q)
1            Ps
1            Qs
```

7. Running HPL on a Single CPU

For now, you will be running HPL on your head node, on a single CPU. Later you will learn how to run HPL over multiple CPUs, each with multiple cores, across multiple nodes.

bash

`# Execute the HPL binary`

```bash
./xhpl
```

> 💡 **Tip**
> When you want to configure and recompile HPL for different architectures, compilers, and systems, adapt the `Make.<NEW_CONFIG>` file and recompile that architecture or configuration.
>
> If your compile fails and you would like to fix your errors and recompile, you must ensure that you reset to a clean start with `make clean`.

🎉 **Congratulations!** You have successfully completed your first HPL benchmark.

Tutorial 3: HPL Concepts Explained

1. What is `mpicc`?

- `mpicc` is the MPI C compiler wrapper.
- MPI stands for Message Passing Interface, which is how HPC programs communicate between processes — potentially across different cores or nodes.
- `mpicc` wraps the regular C compiler (`gcc`) and links it automatically with MPI libraries, so your program can use MPI functions.
- If your system can't find `mpicc`, HPL won't compile properly because it relies on MPI.

That's why we temporarily added it to `PATH`:

/code
export PATH=/usr/lib/x86_64-linux-gnu/openmpi/bin:$PATH
/code

This just tells the shell: "Hey, the `mpicc` binary lives here, check this folder."

---

2. What is `ARCH` in the Makefile?

- `ARCH` is just a label for this particular configuration/build.
- HPL can be compiled in different ways depending on:
  - Your CPU architecture
  - Which math libraries you use
  - How many cores or nodes you want to target
- Each configuration gets its own `ARCH` name (like a version tag).
- This is why in the Makefile you see:

```
ARCH = <YOUR_NAME>
```

It lets HPL organize binaries and settings for this build. Later you could make `ARCH=multi_core` or `ARCH=GPU` if you wanted a different build.

---

3. What are `LAdir` and `LAlib` doing?

- HPL does a lot of matrix math — it needs linear algebra libraries.

- `LAdir` points to the folder where those libraries live (e.g., ATLAS or BLAS/LAPACK).
- `LAlib` tells the compiler which specific library files to link when building HPL.

Example:

```
LAdir = /usr/lib/x86_64-linux-gnu/atlas/
LAlib = $(LAdir)/libblas.so $(LAdir)/liblapack.so
```

This ensures HPL knows how to perform matrix multiplication efficiently.

---

4. Why do we make a separate Makefile / configure HPL?

- HPL isn't a simple "download and run" program — it's a benchmarking tool that needs to know:
  - How to compile on your specific system
  - Which math libraries to use
  - Which MPI compiler to use
- By copying `Make.Linux_PII_CBLAS_gm` to `Make.<YOUR_NAME>` and editing it, we create a custom build configuration for your VM.
- So yes, you are building a custom executable of HPL — this is why the binary ends up in `bin/<YOUR_NAME>/xhpl`.

---

5. Are we making a "temporary" thing?

- Modifying `PATH` to include `mpicc` is **temporary**:
  - It only lasts for your current shell session.
  - Next login, you'd need to add it again (or put it in `.bashrc` for permanent use).
- Everything else (Makefile edits, compilation, `xhpl` binary) is **permanent** for this configuration — you don't need to redo it unless you want a different build.

---

6. Summary of what we did

1. Installed dependencies (compiler, MPI, math libraries).
2. Downloaded HPL source from a working mirror.
3. Copied and edited a Makefile to match Ubuntu + your VM setup.
4. Temporarily updated `PATH` so `mpicc` can be found.
5. Compiled HPL → created a custom `xhpl` binary in `bin/<YOUR_NAME>`.
6. Edited `HPL.dat` to run on 1 process / single CPU.
7. Ran `./xhpl` → validated the benchmark works.

> **Key point:** HPL is highly configurable. You could later compile multiple builds (`ARCH=multi_core`, `ARCH=cluster`) for different setups — that's why we do this Makefile / PATH / LAdir setup.